



MongoDB Developer



MONGODB DEVELOPER CENTER



DEVELOPER TOPICS



LANGUAGES



PYTHON



TUTORIALS

Simplify Semantic Search With LangChain and MongoDB



Brian Leonard

4 min read • Published Sep 23, 2024 • Updated Oct 28, 2024

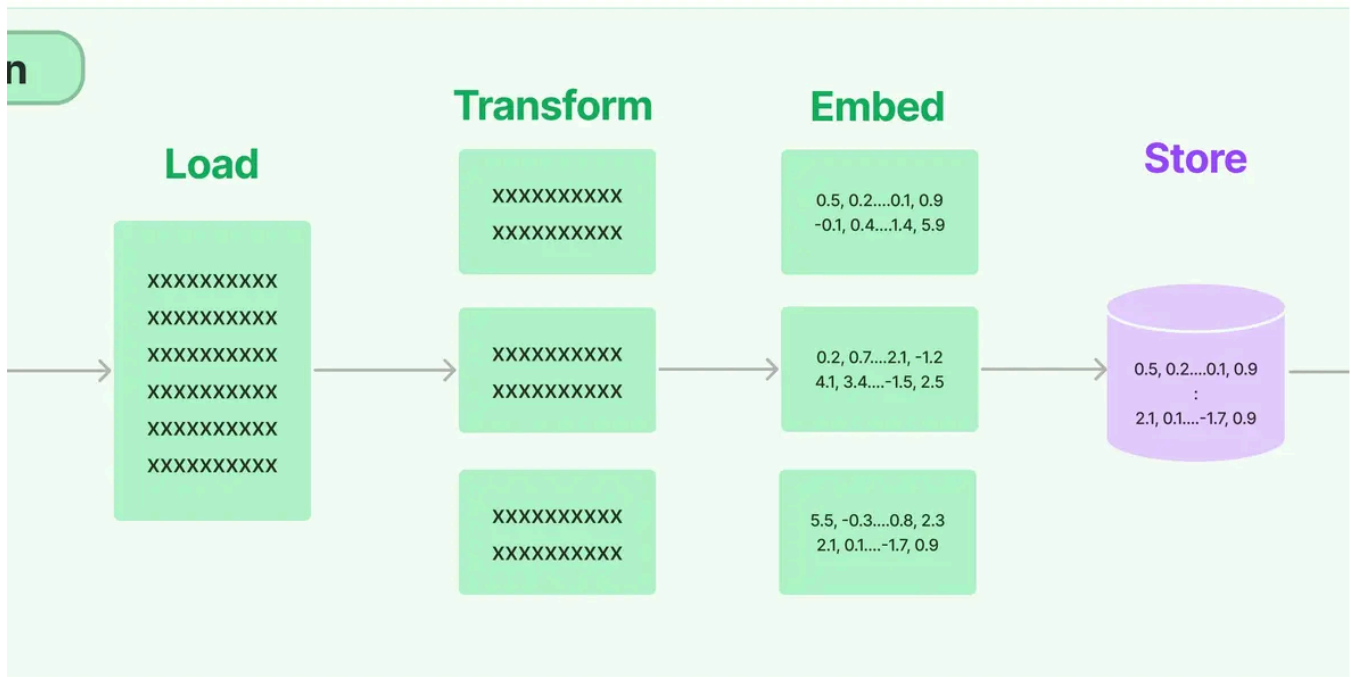
AI

Atlas

Vector Search

Python

FULL APPLICATION



Rate this tutorial

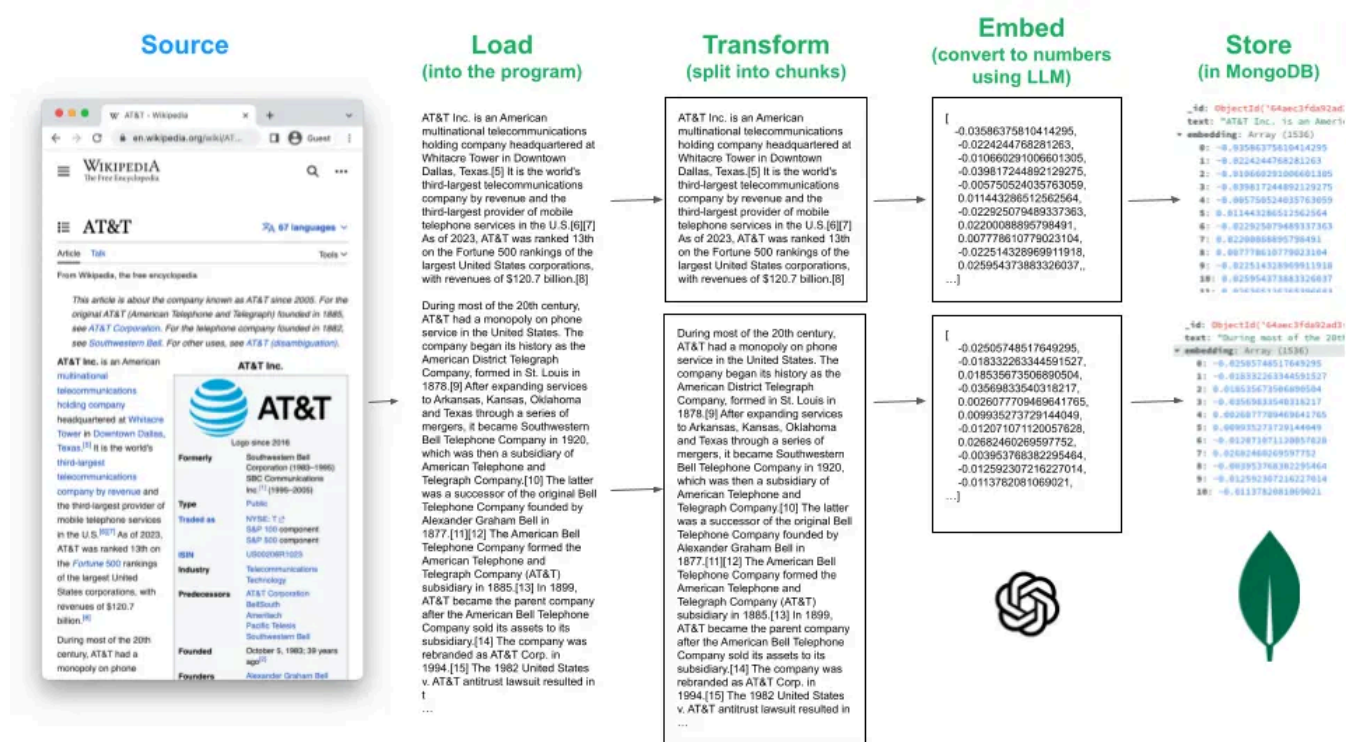


Semantic Search Made Easy With LangChain and MongoDB

Enabling semantic search on user-specific data is a multi-step process that includes loading, transforming, embedding and storing data before it can be queried.

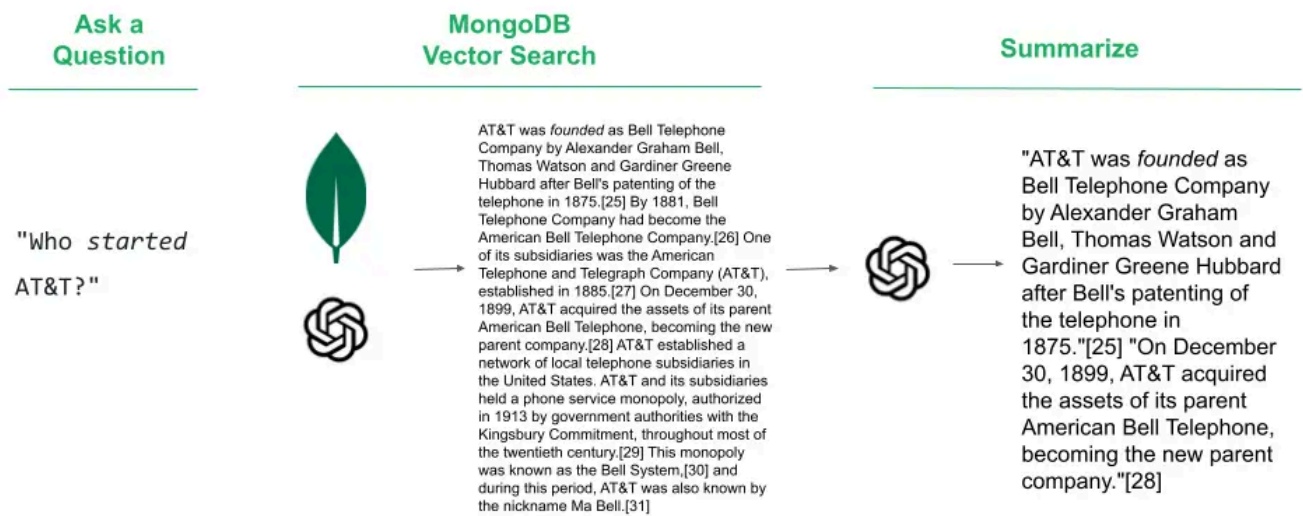
That graphic is from the team over at [LangChain](#), whose goal is to provide a set of utilities to greatly simplify this process.

In this tutorial, we'll walk through each of these steps, using MongoDB Atlas as our Store. Specifically, we'll use the [AT&T](#) and [Bank of America](#) Wikipedia pages as our data source. We'll then use libraries from LangChain to Load, Transform, Embed and Store:



Once the source is store is stored in MongoDB, we can retrieve the data that interests us:

Retrieve



Prerequisites

- [MongoDB Atlas Subscription](#) (Free Tier is fine)
- Open AI [API key](#)

Quick Start Steps

1. Get the code:

```
1 git clone https://github.com/wbleonard/atlas-langchain.git
```



1. Update [params.py](#) with your MongoDB connection string and Open AI [API key](#).

2. Create a new Python environment

```
1 python3 -m venv env
```



1. Activate the new Python environment

```
1 source env/bin/activate
```



1. Install the requirements

```
1 pip3 install -r requirements.txt
```



1. Load, Transform, Embed and Store

```
1 python3 vectorize.py
```



1. Retrieve

```
1 python3 query.py -q "Who started AT&T?"
```



The Details

Load -> Transform -> Embed -> Store

Step 1: Load

There's no lacking for sources of data: Slack, YouTube, Git, Excel, Reddit, Twitter, etc., and [LangChain provides a growing list](#) of integrations that includes this list and many more.

For this exercise, we're going to use the [WebBaseLoader](#) to load the Wikipedia pages for [AT&T](#) and [Bank of America](#).

```
1 from langchain_community.document_loaders import WebBaseLoader
2
3 # Step 1: Load
4 loaders = [
5     WebBaseLoader("https://en.wikipedia.org/wiki/AT%26T"),
6     WebBaseLoader("https://en.wikipedia.org/wiki/Bank_of_America")
```



```

7 ]
8
9 docs = []
10
11 for loader in loaders:
12     for doc in loader.lazy_load():
13         docs.append(doc)

```

Step 2: Transform (Split)

Now that we have a bunch of text loaded, it needs to be split into smaller chunks so we can tease out the relevant portion based on our search query. For this example we'll use the recommended [RecursiveCharacterTextSplitter](#). As I have it configured, it attempts to split on paragraphs (`"\n\n"`), then sentences(`"(?<=\. )"`), then words (`" "`) using a chunk size of 1000 characters. So if a paragraph doesn't fit into 1000 characters, it will truncate at the next word it can fit to keep the chunk size under 1000 characters. You can tune the `chunk_size` to your liking. Smaller numbers will lead to more documents, and vice-versa.

```

1 # Step 2: Transform (Split)
2 from langchain.text_splitter import RecursiveCharacterTextSplitter
3 text_splitter = RecursiveCharacterTextSplitter(chunk_size=1000, chunk_overla
4                                     "\n\n", "\n", r"(?<=\. )", " "
5 docs = text_splitter.split_documents(docs)

```

Step 3: Embed

[Embedding](#) is where you use an LLM to create a vector representation text. There are many options to choose from, such as [OpenAI](#) and [Hugging Face](#), and LangChain provides a standard interface for interacting with all of them.

For this exercise we're going to use the popular [OpenAI embedding](#). Before proceeding, you'll need an [API key](#) for the OpenAI platform, which you will set in [params.py](#).

We're simply going to load the embedder in this step. The real power comes when we store the embeddings in Step 4.

```

1 # Step 3: Embed

```

```
2 from langchain_openai import OpenAIEmbeddings
3 embeddings = OpenAIEmbeddings(openai_api_key=params.OPENAI_API_KEY)
```



Step 4: Store

You'll need a vector database to store the embeddings, and lucky for you MongoDB fits that bill. Even luckier for you, the folks at LangChain have a [MongoDB Atlas module](#) that will do all the heavy lifting for you! Don't forget to add your MongoDB Atlas connection string to [params.py](#).

```
1 # Step 4: Store
2 from pymongo import MongoClient
3 from langchain_mongodb.vectorstores import MongoDBAtlasVectorSearch
4
5 client = MongoClient(params.MONGODB_CONN_STRING)
6 collection = client[params.DB_NAME][params.COLL_NAME]
7
8 # Insert the documents in MongoDB Atlas with their embedding
9 docsearch = MongoDBAtlasVectorSearch.from_documents(
10     docs, embeddings, collection=collection, index_name=index_name
11 )
```



Step 5: Index the Vector Embeddings

The final step before we can query the data is to create a [search index on the stored embeddings](#).

If you're on Atlas dedicated compute, Langchain can do this for you.

```
1 # Step 5: Create Vector Search Index
2 # THIS ONLY WORKS ON DEDICATED CLUSTERS (M10+)
3 docsearch.create_vector_search_index(dimensions=1536, update=True)
```



If you are on shared compute (M0, M2 or M5), in the Atlas console, create a Atlas Vector Search `langchain_vsearch_index` with the following definition:

```
1 {
2     "fields": [
3     {
4         "type": "vector",
5         "path": "embedding",
6         "numDimensions": 1536,
7         "similarity": "cosine"
```



```
8     }  
9   ]  
10 }
```

Create a Vector Search Index



Configuration Method

Select how you would like to build and customize your search index. You can also create, edit, and manage search indexes using the [Atlas API](#).

i NOTE

At this time, search indexes cannot be created for time series collections.

Atlas Search [Learn more](#)

Visual Editor

Learn about index definitions in a more guided experience.

JSON Editor

Edit the raw index definition with an embedded JSON editor.

Atlas Vector Search [Learn more](#)

JSON Editor

Create a vector search index definition with an embedded JSON editor.

Cancel

Next

Create a Vector Search Index



JSON Editor

[View Atlas Vector Search Docs](#)

Search Index Type: `vectorSearch`

Give your vector search index a name for easy reference, and select the database and collection you want to draw data from. You can also create, edit, and manage search indexes using the [Atlas API](#).

Database and Collection

✖ sea

- ▼ search_db
- ☒ search_col

Index Name

langchain_vsearch_index

```
1 {
2   "fields": [
3     {
4       "type": "vector",
5       "path": "embedding",
6       "numDimensions": 1536,
7       "similarity": "cosine"
8     }
9   ]
10 }
```

Back

Cancel

Next

Create a Vector Search Index



Review vector search index "langchain_vsearch_index" for search_db.search_col

[View Atlas Vector Search Docs](#)

```
1 {
2   "fields": [
3     {
4       "type": "vector",
5       "path": "embedding",
6       "numDimensions": 1536,
7       "similarity": "cosine"
8     }
9   ]
10 }
```

Back

Cancel

Create Search Index

You'll find the complete script in [vectorize.py](#), which needs to be run only once or when new data sources are added.

```
1 python3 vectorize.py
```



Retrieve

We could now run a search, using methods like [similarity_search](#) or [max_marginal_relevance_search](#) and that would return the relevant slice of data, which in our case would be an entire paragraph. However, we can continue to harness the power of the LLM to [contextually compress](#) the response so that it more directly tries to answer our question.

```

1  from pymongo import MongoClient
2  from langchain_mongodb.vectorstores import MongoDBAtlasVectorSearch
3  from langchain.embeddings.openai import OpenAIEmbeddings
4  from langchain.llms import OpenAI
5  from langchain.retrievers import ContextualCompressionRetriever
6  from langchain.retrievers.document_compressors import LLMChainExtractor
7
8  # Initialize MongoDB python client
9  client = MongoClient(params.MONGODB_CONN_STRING)
10 collection = client[params.DB_NAME][params.COLL_NAME]
11
12 # initialize vector store
13 vectorStore = MongoDBAtlasVectorSearch(
14     collection, OpenAIEmbeddings(openai_api_key=params.OPENAI_API_KEY), ind
15 )
16 # perform a search between the embedding of the query and the embeddings of
17 print("\nQuery Response:")
18 print("-----")
19 docs = vectorStore.max_marginal_relevance_search(query, K=1)
20 #docs = vectorStore.similarity_search(query, K=1)
21
22 print(docs[0].metadata['title'])
23 print(docs[0].page_content)
24
25 # Contextual Compression
26 llm = OpenAI(openai_api_key=params.OPENAI_API_KEY, temperature=0)
27 compressor = LLMChainExtractor.from_llm(llm)
28
29 compression_retriever = ContextualCompressionRetriever(
30     base_compressor=compressor,
31     base_retriever=vectorStore.as_retriever()
32 )

```

```

1  python3 query.py -q "Who started AT&T?"
2
3  Your question:
4  -----
5  Who started AT&T?
6
7  AI Response:
8  -----
9  AT&T - Wikipedia
10 "AT&T was founded as Bell Telephone Company by Alexander Graham Bell, Thoma

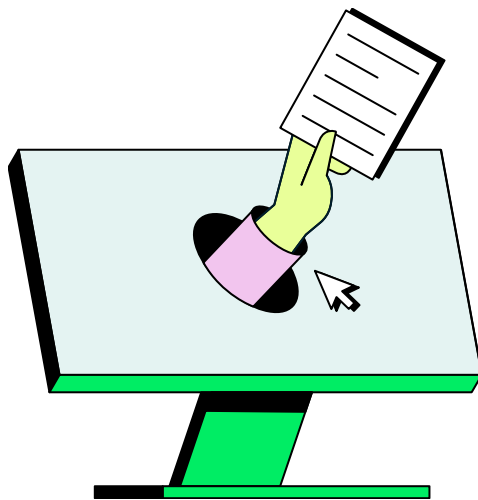
```

Resources

- [MongoDB Atlas](#)

- [Open AI API key](#)
- [LangChain](#)
 - [WebBaseLoader](#)
 - [RecursiveCharacterTextSplitter](#)
 - [MongoDB Atlas module](#)
 - [Contextual Compression.](#)
 - [MongoDBAtlasVectorSearch API](#)

Top Comments in Forums



There are no comments on this article yet.

[Start the Conversation](#)

Rate this tutorial

Related

TUTORIAL

RAG Evaluation: Detecting Hallucinations With Patronus AI and MongoDB

Aug 15, 2024 | 11 min read

QUICKSTART

Quick Start 2: Vector Search With MongoDB and OpenAI

May 06, 2024 | 12 min read

TUTORIAL

Getting Started With MongoDB and AWS CodeWhisperer

Sep 26, 2024 | 3 min read

TUTORIAL

Getting Started With MongoDB Atlas Serverless, AWS CDK, and AWS Serverless Computing

Aug 09, 2024 | 18 min read

[Request a Tutorial](#)

About

Careers

Investor Relations

Legal

GitHub

Security Information

Trust Center

Connect with Us

Deployment Options

MongoDB Atlas

Enterprise Advanced

Community Edition

Support

Contact Us

Customer Portal

Atlas Status

Customer Support

Data Basics

Vector Databases

NoSQL Databases

Document Databases

RAG Database

ACID Transactions

MERN Stack

MEAN Stack

